

Backend API Creation - Documentation

1. Overview

This test evaluates the student's ability to design and implement RESTful backend APIs with authentication, validation, database integration, and structured responses.

2. Tech Requirements

- Framework: ASP.NET Core / JS
- Database: SQL or NoSQL
- Authentication: JWT Based
- API Format: REST
- Response Format: JSON

3. Database Schema

Students Table

- id (int, PK)
- firstName (varchar(50))
- lastName (varchar(50))
- email (varchar(50), unique)
- mobile (varchar(50))
- course (varchar(50))

Users Table

- id (int, PK)
- email (varchar(50))
- password (varchar(50))
- profileimg (varchar(max)) | null

Courses Table

- id (int, PK)
- name (varchar(50))
- duration (varchar(50))
- fees (decimal (8,2))

4. Base URL

<http://localhost:5000/api/>

5. Authentication Module

Endpoint: POST /auth/login

Request Body:

```
{  
  "email": "student@test.com",
```

```
"password": "123456"
}
```

Success Response:

```
{
  "status": true,
  "token": "jwt_token_here",
  "message": "Login successful"
}
```

Failure Response:

```
{
  "status": false,
  "message": "Invalid credentials"
}
```

6. Student Module

5.1 Create Student

POST /students

Headers: Authorization: Bearer <token>

Request:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@test.com",
  "mobile": "9876543210",
  "course": "BCA"
}
```

Success Response:

```
{
  "status": true,
  "message": "Student created",
  "data": { "id": 1 }
}
```

5.2 Get All Students

GET /students

Headers: Authorization: Bearer <token>

Response:

```
{
  "status": true,
  "data": [
    {
      "id": 1,
      "firstName": "John",

```

```
"lastName": "Doe",  
"email": "john@test.com",  
"mobile": "9876543210",  
"course": "BCA"  
}  
]  
}
```

5.3 Get Student By ID
GET /students/{id}

5.4 Update Student
PUT /students/{id}

5.5 Delete Student
DELETE /students/{id}

7. Course Module

6.1 Create Course
POST /courses

6.2 Get Courses
GET /courses

6.3 Update Course
PUT /courses/{id}

6.4 Delete Course
DELETE /courses/{id}

8. Validation Rules

- Email must be valid format
- Mobile must be 10 digits
- Required fields cannot be null
- Password must be minimum 8 characters with password policy(Minimum 8 characters, At least 1 uppercase, At least 1 lowercase, At least 1 number, At least 1 special character)

9. Status Codes

200 = Success
201 = Created
400 = Bad Request
401 = Unauthorized
404 = Not Found
500 = Server Error

10. Advanced / Complex Endpoints

11.1 Search Students (Filtering + Pagination + Sorting)

GET /students/search?page=1&pageSize=10&course=BCA&sortBy=firstName&order=asc

Response:

```
{
  "status": true,
  "page": 1,
  "pageSize": 10,
  "totalRecords": 120,
  "data": []
}
```

11.2 Bulk Insert Students (File Upload & JSON Array Pass (Both in different endpoints))

POST /students/bulk

11.3 Assign Course to Student

POST /students/{id}/assign-course/{courseId}

11.4 Get Students With Course Details (JOIN)

GET /students/with-courses

11.5 Soft Delete Student

PATCH /students/{id}/deactivate

11.6 Restore Student

PATCH /students/{id}/restore

11.7 Get Activity Logs

GET /logs/students/{id}

11.8 Export Students

GET /students/export?format=excel | pdf | csv

11. Evaluation Criteria

- Clean Architecture
- Proper Folder Structure
- JWT Authentication Implementation
- Validation Handling
- Proper Status Codes
- Error Handling
- Database Integration
- Naming Conventions

- Code Readability
- Scalability Consideration

12. Additional Evaluation Criteria (Advanced Level)

- Uses Repository + Service Layer Pattern
- Implements DTO Mapping
- Uses Async/Await correctly
- Implements Global Exception Middleware
- Uses Transactions where required
- Proper Indexing in DB
- Caching (optional bonus)
- API Rate Limiting (bonus)
- Logging System (Serilog + Seq)